Réalisé par :
- Ali EL HASSNAOUI.
- TCHINDE SINGHE RODERICK DARELL

Encadré par : Mme Clotilde Toullec.

Red5-Server - Projet de Qualité Logicielle

1. Présentation globale du projet

1.1 Objectif de l'étude

L'objectif principal est d'évaluer la qualité logicielle du projet **Red5-Server**, un serveur multimédia open-source développé en Java. L'analyse vise notamment à :

- comprendre l'organisation générale du logiciel ;
- étudier son architecture et la répartition de ses composants ;
- observer sa documentation et son historique de maintenance ;
- identifier des indicateurs de qualité ou de dette technique ;
- dégager des pistes d'amélioration pertinentes pour la seconde phase du projet.

Cette démarche permet de dépasser une simple lecture descriptive du code pour proposer une analyse critique de sa maintenabilité et de sa robustesse.

1.2 Utilité du projet

Red5-Server est un serveur multimédia orienté streaming et communication temps réel. Il est principalement utilisé pour la diffusion de contenus audio/vidéo et la gestion d'échanges interactifs via différents protocoles de communication.

Parmi les fonctionnalités que l'on peut associer au projet, on retrouve notamment :

- le **streaming en direct** et à la demande ;
- la prise en charge de protocoles comme **RTMP**, **HLS**, **WebSocket** et, selon la documentation du projet, des mécanismes liés au temps réel ;
- l'enregistrement de flux côté serveur ;
- la possibilité de servir de base à des applications multimédias plus riches, comme des conférences en ligne, des outils interactifs ou des échanges audio/vidéo.

Le projet se présente donc comme une infrastructure serveur spécialisée dans le traitement de flux multimédias, avec une forte dimension réseau et temps réel.

1.3 Lancement et production logicielle

Le projet s'appuie sur **Maven** pour sa compilation. Une commande de build telle que :

```
mvn -Dmaven.test.skip=true install
```

permet de générer les artefacts principaux du système.

Cependant, un point important ressort de notre analyse : **les seules informations présentes dans le README principal ne nous ont pas permis, à elles seules, d'obtenir un lancement satisfaisant du logiciel**. En pratique, d'autres fichiers de documentation présents dans le dépôt, notamment `claude.md`, apportent des précisions plus opérationnelles sur les commandes à exécuter et sur les prérequis système.

Le résultat de la compilation produit principalement des archives Java de type **JAR**, correspondant aux différents modules du serveur.

1.4 Première appréciation globale

Dès cette première lecture, Red5-Server apparaît comme un projet **ancien, dense et techniquement riche**, construit autour d'un domaine complexe : le streaming multimédia temps réel. Cette nature explique déjà plusieurs caractéristiques que l'on retrouvera plus loin dans le rapport : classes volumineuses, logique protocolaire bas niveau, complexité cyclomatique élevée sur certaines portions, et présence de constantes métier fortement liées au domaine.

---

2. Description et documentation du projet

2.1 Présence d'une documentation

Le dépôt contient bien un **README principal**, mais également plusieurs autres fichiers Markdown situés à différents endroits de l'arborescence. Cela suggère une documentation répartie entre une présentation générale du projet et des documents plus spécialisés, probablement propres à certains sous-modules ou à certains usages.

Cette multiplicité n'est pas un défaut en soi, mais elle pose une question de lisibilité : **où se trouve réellement la documentation de référence pour installer, construire et lancer le projet ?**

2.2 Pertinence du README principal

Le README principal donne une vue d'ensemble utile du projet, de ses modules et de son organisation. En revanche, il ne centralise pas nécessairement toutes les informations pratiques nécessaires au lancement réel du logiciel. Une partie des commandes utiles semble plutôt documentée dans `claude.md`, qui nous a paru plus directement exploitable pour le build et les essais de prise en main.

Autrement dit, le README principal est **pertinent pour comprendre le projet**, mais **pas totalement suffisant pour le mettre en œuvre sans ambiguïté**.

2.3 Mise à jour de la documentation

Nous avons relevé que la dernière modification du README principal remonte au **19/02/2025**. Ce point est intéressant pour l'analyse : la documentation n'est donc pas abandonnée, mais les messages de commit associés restent très génériques, par exemple sous la forme de **"update readme"**, ce qui ne permet pas de comprendre précisément la nature des changements effectués.

Cela réduit la traçabilité documentaire : le dépôt montre qu'une mise à jour existe, mais pas clairement **pourquoi** ni **sur quel aspect** elle a été faite.

2.4 Diagnostic documentaire

La documentation du projet est globalement **réelle, riche et utile**, mais elle souffre d'un léger manque de centralisation.

On peut donc en tirer le constat suivant :

- le projet n'est pas dépourvu de documentation ;
- certaines informations pratiques existent bien et sont exploitables ;
- en revanche, la répartition entre plusieurs fichiers peut rendre la prise en main moins immédiate ;
- le README gagnerait à devenir le point d'entrée unique, avec renvoi structuré vers les documents secondaires.

---

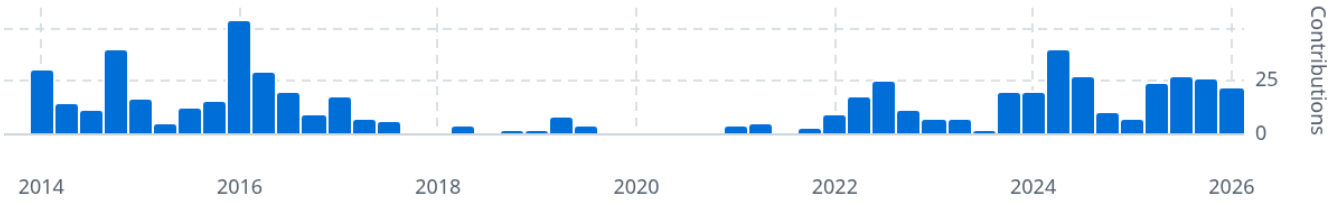
3. Historique du logiciel et activité du dépôt

3.1 Composition de l'équipe

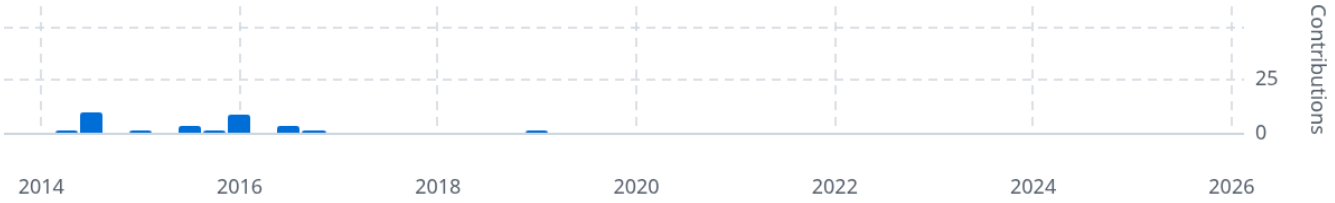
L'analyse du dépôt Git montre que le projet a mobilisé un nombre non négligeable de contributeurs. Une première lecture fait apparaître **17 contributeurs** au total. La répartition n'est toutefois pas homogène : une grande partie des contributeurs n'a effectué qu'un très petit nombre de commits, tandis qu'un noyau plus restreint concentre l'essentiel de l'activité.

Parmi les contributeurs les plus visibles figurent notamment **Mondain**, **copilot** et **solomax**, avec une domination particulièrement nette de **Mondain** sur la durée.

mondain's Commits



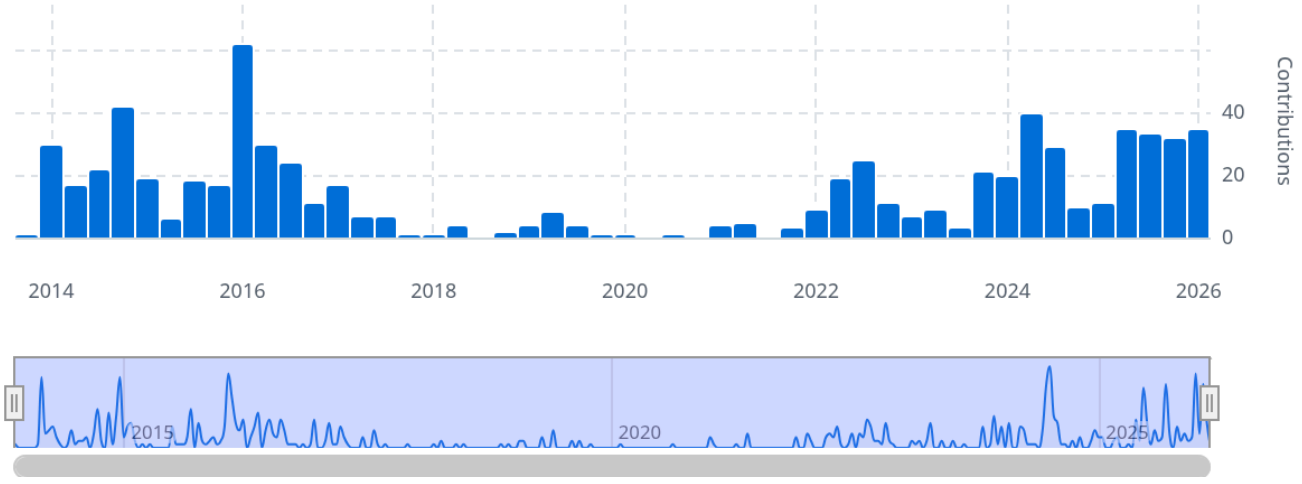
solomax's Commits



3.2 Régularité de l'activité

Commits over time

Weekly from Nov 17, 2013 to Mar 1, 2026



Le projet apparaît **toujours actif**. Le dernier commit mentionné dans notre analyse remonte au **19 février 2026**, ce qui, au moment de l'étude, indique une activité récente.

Cela étant, cette activité récente ne signifie pas nécessairement un rythme élevé. Sur les trente derniers jours, nous ne relevons qu'un nombre limité de commits, ce qui peut s'interpréter de plusieurs façons :

- le projet est stable et nécessite moins d'interventions ;
- les évolutions sont plus ponctuelles ;
- ou encore la maintenance se concentre sur des ajustements ciblés plutôt que sur une évolution intensive.

3.3 Branches

Défaut

Bifurquer	Mis à jour	Vérifier l'état	Derrière	Devant	Demande de fusion
`principal`	il y a 2 semaines	1 / 1	Défaut		

Branches actives

Bifurquer	Mis à jour	Vérifier l'état	Derrière	Devant	Demande de fusion
`bogue/324`	il y a 2 mois		52	2	
`bug/hash-mash`	il y a 3 mois		215	14	

analyse:

- on dénombre environ **10 branches** au total ;
- **deux branches** semblent encore actives, nommées `bug` et `bug-hash` ;
- l'activité sur ces branches concerne surtout un nombre restreint de contributeurs.

Cette situation suggère un usage des branches plutôt **ciblé et maîtrisé**, davantage orienté correctifs que développement parallèle massif.

3.4 Pull requests

Le dépôt montre un usage important du mécanisme de **pull request**, avec un volume élevé de PR au cours de la vie du projet. C'est généralement un bon signe sur le plan qualité : cela traduit un processus de contribution structuré, une validation explicite des changements et, vraisemblablement, une forme de revue de code.

3.5 Lecture globale de l'historique

L'historique du projet donne l'image d'un logiciel **mature**, maintenu sur la durée, mais dont la dynamique repose surtout sur un petit noyau de développeurs.

Open Issues 4,437 See history

client	300
common	1,439
io	1,621
server	727
service	112
tests	236

4. Architecture logicielle

4.1 Utilisation de bibliothèques extérieures

Le projet fait appel à plusieurs bibliothèques Java majeures, cohérentes avec son domaine applicatif. On retrouve notamment des technologies comme **Spring Framework**, **Apache Mina**, **Tomcat**, **SLF4J** et **Logback**.

Le recours à ces bibliothèques est justifié : un serveur de streaming temps réel doit gérer des problématiques de communication réseau, de sérialisation, de journalisation, de configuration et de cycle de vie applicatif qu'il serait coûteux de réimplémenter entièrement.

Cette dépendance à des frameworks externes n'est donc pas un signe de faiblesse, mais un marqueur d'industrialisation du projet.

4.2 Organisation en paquetages

L'analyse du code de production fait apparaître environ **125 paquetages**. L'arborescence est relativement structurée et laisse apparaître de grands ensembles comme : *io core serveur*,*service* ...

Cette organisation laisse entrevoir une séparation par responsabilités : cœur serveur, , logique cliente, flux, plugins,
Les noms de paquetages révèlent aussi plusieurs intentions architecturales. Des termes comme `plugin`, `messaging`, `stream`, `codec` ou `protocol` montrent un découpage technique fortement orienté domaine.

4.3 Répartition des classes

Classes 765 See history

client	49
common	306
io	249
server	156
service	5
tests	—

Le projet compte environ **766 classes Java** au total, auxquelles s'ajoutent **plus de 6000 méthodes** (en incluant les méthodes classiques, les constructeurs et certains accesseurs générés). Ces chiffres indiquent clairement que Red5-Server est un projet de **grande taille**, avec une base de code importante et une structure logicielle relativement dense.

Rapporté aux **125 paquetages** identifiés précédemment, cela correspond à une moyenne d'environ **6 classes par paquetage**, ce qui reste globalement raisonnable et suggère une organisation modulaire correcte. Cependant, cette moyenne masque une réalité plus contrastée : certains paquetages restent très compacts tandis que d'autres, notamment ceux liés aux **protocoles de streaming, au décodage, aux codecs et à la gestion réseau**, concentrent une part beaucoup plus importante de classes et surtout de logique métier.

Le nombre élevé de méthodes confirme également que le système implémente de nombreux comportements liés au traitement des flux, à la gestion des connexions et au décodage des protocoles multimédias. Cette richesse fonctionnelle est cohérente avec la nature du projet, mais elle augmente aussi mécaniquement la **complexité potentielle du code**, ce qui peut rendre certaines parties du système plus difficiles à maintenir ou à tester.

4.4 Organisation des classes et cohésion

La hiérarchie de classes ne semble pas excessivement profonde. L'héritage est présent, mais globalement maîtrisé, souvent au travers d'interfaces métier importantes comme `IConnection` ou `IStream`.

Sur le plan de la cohésion, certains paquetages spécialisés apparaissent assez cohérents, par exemple ceux dédiés aux formats ou à l'I/O. En revanche, les composants centraux qui orchestrent plusieurs responsabilités ont naturellement tendance à devenir plus dispersés, car ils servent d'articulation entre plusieurs sous-systèmes.

4.5 Lecture architecturale générale

L'architecture de Red5-Server donne l'image d'un système **modulaire mais dense**, organisé autour de couches techniques nettes. Cette structuration constitue un point fort du projet. En revanche, comme souvent dans les systèmes anciens et riches fonctionnellement, certains composants centraux absorbent progressivement plus de responsabilités que prévu.

---

5. Analyse approfondie de la qualité

5.1 Tests

Le projet contient environ **66 classes de test** pour **773 classes** metier, ainsi que **plus de 6000 méthodes** si l'on inclut les constructeurs et les autres membres comptabilisés par les outils d'analyse. De plus on a que 26% des classes testées et 18 % des methodes pour un projet de cettte envergure c'est clairement très peu .Ce volume confirme que Red5-Server est un projet de taille conséquente, avec une base de code large et structurellement dense. En rapportant les tests au nombre de classes de production, on obtient un ordre de grandeur d'environ **une classe de test pour douze classes métier**, ce qui reste relativement faible pour un système aussi vaste. Cette observation suggère que l'effort de validation automatisée existe, mais qu'il demeure limité au regard de la complexité potentielle du projet et du nombre important de comportements à couvrir.

Les tests présents semblent surtout couvrir des comportements unitaires ou techniques, en particulier sur des aspects d'I/O ou de logique locale.

Concernant la couverture, une valeur de **0 %** a été observée dans le rapport disponible. cela est du à une mauvaise configuration du pom qui empeche notre jacoco d'effectuer une analyse correcte sous sonar .neamois
un autre logiciel de mesure nous revèle in pourcentage de couverture de test d'environs 56% ce qui est très faible

Element	Class, %	Method, %	Line, %	Branch, %
all	26% (188/703)	18% (1029/5664)	15% (4421/29333)	9% (1422/14557)
org.red5	26% (188/702)	18% (1029/5663)	15% (4421/29332)	9% (1422/14557)
Version	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

Interprétation :

- la base de tests existe, mais paraît limitée au regard de la taille du code ;

5.2 Commentaires

L'analyse fait ressortir environ :

- **73 358 lignes de code** ;
- **50 514 lignes de commentaires** ;
- soit un pourcentage proche de **68,8 %**.

Cette proportion est très élevée. Elle traduit un effort de documentation important, probablement renforcé par la présence de Javadoc, de commentaires techniques et de traces historiques dans le code.

Il faut toutefois nuancer cette bonne impression : une forte quantité de commentaires n'implique pas automatiquement une bonne qualité documentaire. La vraie question est celle de la **pertinence** et de la **mise à jour** des commentaires. Malgré cela, l'ensemble laisse une impression favorable : le projet n'est pas un code totalement opaque.

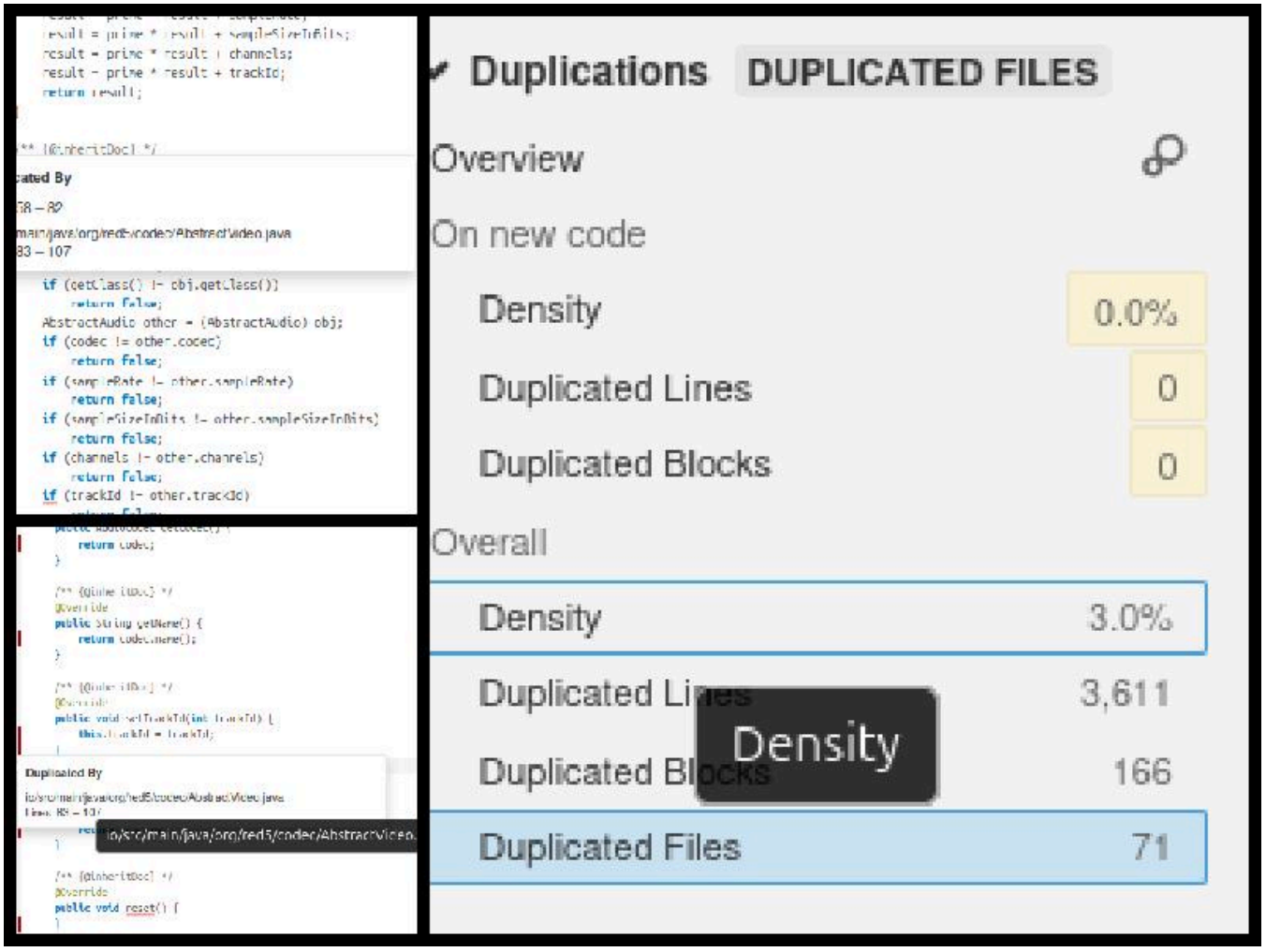
5.3 Dépréciation

Comme dans beaucoup de projets Java anciens, on observe la présence de mécanismes dépréciés, notamment en lien avec l'évolution de certaines bibliothèques externes(source:google). Des méthodes marquées `@Deprecated` subsistent, ce qui peut s'expliquer par la nécessité de préserver une compatibilité descendante ou par l'inertie naturelle d'un logiciel mature.

La dépréciation n'est donc pas ici un défaut isolé, mais le signe d'une longue durée de vie technique.

5.4 Duplication de code

D'après l'analyse réalisée avec **SonarQube**, la duplication de code concerne **71 fichiers** du projet. L'outil détecte également **166 blocs de code dupliqués**, ce qui représente environ **3 % de densité de duplication** dans l'ensemble du code.



Ces chiffres montrent que la duplication reste **relativement limitée** à l'échelle d'un projet comportant plus de **700 classes et plus de 6000 méthodes**. Une densité de 3 % reste généralement considérée comme acceptable dans des systèmes logiciels de grande taille. Cependant, la présence de plus de cent blocs dupliqués indique tout de même que certaines portions du code reproduisent des logiques similaires.

Dans de nombreux cas, cette duplication ne provient pas forcément d'un simple copier-coller massif, mais plutôt d'une **répétition progressive de structures logiques similaires**, notamment dans les parties du système liées au traitement des protocoles réseau, au décodage des flux ou à la gestion des connexions.

.

5.5 God Classes

Plusieurs classes se distinguent clairement par leur taille ou leur densité fonctionnelle.

`PlayEngine.java`

Cette classe atteint environ **1449 lignes** et plus de **64 méthodes**. Elle semble concentrer une part importante de la logique liée à la lecture, à la temporalité, à l'état et à l'orchestration du flux. Le simple nom *Engine* suggère déjà un rôle centralisateur.

`RTMPConnection.java`

Cette classe est encore plus révélatrice : environ **1152 lignes** et **125 méthodes**. Elle constitue manifestement un point névralgique du système, ce qui explique en partie sa taille, mais confirme aussi son caractère potentiellement difficile à maintenir.

`OBUParser.java`

Avec environ **1696 lignes**, cette classe est très volumineuse. Son cas demande toutefois une lecture plus prudente : dans certains cas, par exemple pour analyser un protocole, une classe très grande peut être acceptable si elle contient une logique technique très spécifique.

Interprétation : ces classes ne représentent qu'une petite part du volume global, mais leur taille absolue suffit à en faire des points sensibles du projet. Dans un système de cette ampleur, ce sont souvent elles qui concentrent les risques de régression et les difficultés d'évolution.

5.6 Analyse des méthodes

Les méthodes les plus complexes se situent logiquement dans les zones liées au décodage de protocole, à la lecture de flux et aux mécanismes réseau.

La classe `RTMPProtocolDecoder.java`, par exemple, présente naturellement une forte complexité cyclomatique en raison de la logique de dispatch, des branchements et des traitements dépendants des octets lus.

Dans ce type de système, une complexité élevée n'est pas toujours évitable. En revanche, elle devient problématique lorsqu'elle se combine avec :

- un grand nombre de responsabilités dans une même méthode ;
- des effets de bord sur l'état d'objets partagés ;
- une testabilité faible ;
- une lisibilité dégradée.

C'est précisément ce risque que l'on observe sur certaines parties centrales de Red5-Server.

5.7 Complexité cyclomatique et complexité cognitive

La **complexité cyclomatique** mesure le **nombre de chemins d'exécution possibles dans un programme**. Plus il y a de conditions, de boucles ou de branches, plus cette complexité augmente.

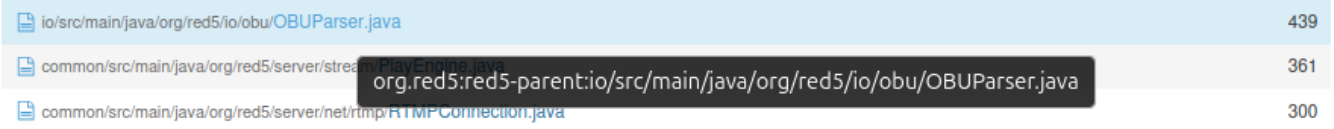
La **complexité cognitive**, quant à elle, mesure **la difficulté pour un développeur de comprendre le code**. Cette métrique pénalise notamment :

- l'imbrication importante des structures (*nested code*) ;
- les structures de contrôle complexes ;
- les logiques difficiles à suivre ou à lire.

!

Complexité cyclomatique

D'après l'analyse réalisée avec Sonar, les fichiers présentant la **plus forte complexité cyclomatique** sont :



- **OBUParser.java** avec une complexité de **439** ;
- **PlayEngine.java** avec **361** ;
- **RTMPConnection.java** avec **300**.

Le cas de **OBUParser.java** peut être partiellement expliqué par la nature même de la classe. Le terme *Parser* indique qu'elle est responsable de l'analyse d'un format ou d'un protocole. Ce type de composant doit souvent gérer de nombreux cas, structures conditionnelles et traitements différents. Une complexité élevée est donc relativement attendue dans ce type de classe.

En revanche, **PlayEngine** et **RTMPConnection** ont également été identifiées précédemment comme des **God Classes potentielles**. Leur complexité importante peut donc être liée au fait qu'elles concentrent plusieurs responsabilités dans un même fichier. Une décomposition de ces classes en composants plus spécialisés pourrait permettre de réduire cette complexité lors d'une phase d'amélioration ultérieure.

Un autre point important est que **les trois fichiers les plus complexes représentent à eux seuls environ 8 % de la complexité cyclomatique totale**, calculée sur environ **660 fichiers analysés**. Cela montre que la complexité du projet est **fortement concentrée dans un nombre limité de classes centrales**.

Complexité cognitive

io/src/main/java/org/red5/io/obu/OBUParser.java	727
common/src/main/java/org/red5/server/stream/PlayEngine.java	716
io/src/main/java/org/red5/io/mp4/impl/MP4Reader.java	437

Concernant la **complexité cognitive**, les fichiers les plus concernés sont :

- **OBUParser.java** ;
- **PlayEngine.java** ;
- **MP4Reader.java**.

Dans cette métrique également, **PlayEngine** apparaît comme un point particulièrement sensible. Sa valeur atteint environ **716**, tandis que le troisième fichier présente une valeur d'environ **437**, soit presque la moitié de celle de PlayEngine.

Cette observation confirme que certaines classes jouent un rôle central dans le système et concentrent une grande partie de la logique métier. Comme pour la complexité cyclomatique, une **répartition plus équilibrée des responsabilités** dans plusieurs classes pourrait améliorer la lisibilité et la maintenabilité du code.

De manière générale, ces résultats montrent que **la complexité globale du projet n'est pas uniformément répartie**, mais plutôt concentrée dans quelques composants clés du système. Ce phénomène est fréquent dans les logiciels de grande taille et constitue un indicateur utile pour identifier les zones qui pourraient être ciblées lors d'une future phase d'amélioration.

6. Nettoyage de code et code smells

6.1 Règles de nommage

Dans l'ensemble, le projet respecte plutôt bien les conventions Java classiques. Les noms liés au domaine — `stream`, `packet`, `decoder`, `session`, `message`, `connection` — sont globalement parlants et cohérents avec le contexte applicatif.

Cependant, les outils d'analyse ont relevé plusieurs anomalies.

Constantes et conventions

```
private static final boolean isDebug = log.isDebugEnabled();
```

Rename this constant name to match the regular expression '^[A-Z][A-Z0-9]*(_[A-Z0-9]+)*$'. 2 years ago L103

Why is this an issue?

Code Smell Critical Open Not assigned 2min effort Comment convention

Nous avons relevé environ **106 issues** de type *“Constant names should comply with a naming convention”*. Toutes ne semblent pas également graves, mais elles montrent qu’une partie des constantes n’est pas nommée de façon homogène.

Problèmes d’héritage (shadowing de champs)

Un autre problème identifié concerne les cas où une classe fille redéclare un champ déjà présent dans sa classe parente. Ce phénomène est appelé **shadowing**.

- Environ **34 issues** de type *“Child class fields should not shadow parent class fields”* ont été relevées.

Ce problème peut rendre le code plus difficile à comprendre, car il devient moins clair de savoir **quel champ est réellement utilisé** (celui de la classe parente ou celui de la classe fille). Cela peut aussi provoquer des erreurs lors de la maintenance du code.

```
ublic class RTMPSClient extends RTMPClient {

    private static String[] cipherSuites;

    // I/O handler
    private RTMPSClientIoHandler ioHandler;
```

ioHandler" is the name of a field in "RTMPClient". Why is this an issue? 8 months ago L48
Code Smell Blocker Open Not assigned 5min effort Comment confusing

Ambiguïtés et collisions de noms

D’autres remarques concernent :

- des appels ambigus à des méthodes de super-classe depuis des classes internes ;
- des noms de classes qui entrent en collision avec ceux d’interfaces ou de super-classes ;
- certaines déclarations peu lisibles où le type final ou l’intention réelle devient difficile à saisir rapidement.

Red5 / io/src/main/java/org/red5/io/amf/Input.java
Rename this class. Why is this an issue? 4 years ago L52
Code Smell Critical Open Not assigned 5min effort Comment pitfall

Red5 / io/src/main/java/org/red5/io/amf/Output.java
Rename this class. Why is this an issue? 4 years ago L49
Code Smell Critical Open Not assigned 5min effort Comment pitfall

Red5 / io/src/main/java/org/red5/io/amf3/Input.java
Rename this class. Why is this an issue? 2 years ago L54
Code Smell Critical Open Not assigned 5min effort Comment pitfall

Red5 / io/src/main/java/org/red5/io/amf3/Output.java
Rename this class. Why is this an issue? 2 years ago L45
Code Smell Critical Open Not assigned 5min effort Comment pitfall

4 of 4 shown

Dans l’ensemble, le nommage reste donc **majoritairement cohérent**, mais il existe des zones localisées où la qualité baisse nettement.

6.2 Nombres magiques

Le projet contient un grand nombre de signalements liés aux nombres magiques. Une valeur brute comme **1878 issues** a été observée, mais elle doit être interprétée avec précaution, car l'outil inclut aussi des cas peu pertinents, comme certains accès à des indices de tableaux.

Les nombres les plus intéressants sont ceux qui portent une signification métier ou protocolaire.

Dans cette catégorie, l'outil d'analyse détecte un grand nombre de **nombres magiques** (valeurs numériques utilisées directement dans le code sans constante explicite). Ces valeurs peuvent correspondre à des paramètres techniques, à des tailles de buffers ou encore à des valeurs propres à certains protocoles.

La figure associée montre plusieurs exemples détectés par l'outil d'analyse. Certaines valeurs ont une signification claire liée au protocole ou au fonctionnement interne du système. Dans ce cas, elles ne sont pas nécessairement incorrectes, mais elles pourraient être rendues plus explicites à l'aide de constantes nommées.

Dans le cadre de ce rapport, l'objectif est uniquement **d'identifier et d'expliquer ces occurrences**, sans modifier le code du projet. L'image associée présente quelques exemples représentatifs de ces valeurs numériques. Par exemple, certaines valeurs correspondent au **port standard du protocole RTMP utilisé pour le streaming audio/vidéo**, d'autres à la **longueur du digest utilisé lors du handshake RTMP**, et d'autres encore à la **taille d'un buffer temporaire utilisé pour traiter les headers ou les paquets du protocole**. Ces nombres ne sont donc pas forcément incorrects : ils ont souvent une **signification technique précise** dans le fonctionnement du système. L'analyse consiste donc simplement à expliquer ce que ces valeurs représentent et pourquoi elles apparaissent dans le code. Dans une phase ultérieure du projet, elles pourraient être remplacées par des constantes explicites afin d'améliorer la lisibilité et la maintenabilité du

code.

```
for (RemotingHeader header : hdr) {
    Output.putString(result, header.getName());
    result.put(header.getMustUnderstand() ? (byte) 0x01 : (byte) 0x00);

    IoBuffer tmp = IoBuffer.allocate(1024);
```

```
 *            Method parameters
 * @return Byte buffer with data to perform remoting call
 */
private IoBuffer encodeInvoke(String method, Object[] params) {
    log.debug("RemotingClient encodeInvoke - method: {} params: {}", method, params);
    IoBuffer result = IoBuffer.allocate(1024);
```

Assign this magic number 1024 to a well-named constant, and use the constant instead. 4 years ago ▾ L101

Why is this an issue?

Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment brain-overload ▾

```
    result.setAutoExpand(true);
    //force version 3
    result.putShort((short) 3);
```

Assign this magic number 3 to a well-named constant, and use the constant instead. 4 years ago ▾ L104

Why is this an issue?

Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment brain-overload ▾

```
    // Headers
    Collection<RemotingHeader> hdr = headers.values();
    result.putShort((short) hdr.size());
    for (RemotingHeader header : hdr) {
        Output.putString(result, header.getName());
        result.put(header.getMustUnderstand() ? (byte) 0x01 : (byte) 0x00);

        IoBuffer tmp = IoBuffer.allocate(1024);
```

Assign this magic number 1024 to a well-named constant, and use the constant instead. 4 years ago ▾ L112

Bilan : tous les nombres magiques ne sont pas de mauvaises pratiques au même degré. Dans Red5-Server, beaucoup relèvent du domaine protocolaire. Le vrai enjeu est donc moins leur suppression que leur **mise en explicitation**.

6.4 Code mort

On remarque la présence de code mort sous plusieurs formes dans ce projet. On peut notamment citer :

6.4.1 Types inutilisés

Red5 / io/.../main/java/org/red5/io/flv/meta/IMetaData.java

K is not used in the interface. Why is this an issue? 4 years ago L22
Code Smell Major Open Not assigned 5min effort Comment cert, unused

V is not used in the interface. Why is this an issue? 4 years ago L22
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/.../main/java/org/red5/io/flv/meta/MetaCue.java

K is not used in the class. Why is this an issue? 4 years ago L31
Code Smell Major Open Not assigned 5min effort Comment cert, unused

V is not used in the class. Why is this an issue? 4 years ago L31
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/.../main/java/org/red5/io/flv/meta/MetaData.java

K is not used in the class. Why is this an issue? 4 years ago L27
Code Smell Major Open Not assigned 5min effort Comment cert, unused

V is not used in the class. Why is this an issue? 4 years ago L27
Code Smell Major Open Not assigned 5min effort Comment cert, unused

6 of 6 shown

Un autre signal intéressant concerne les *unused type parameters*. Deux interprétations sont possibles :

- soit il s'agit d'un résidu de conception, devenu inutile avec l'évolution du code ;
- soit il s'agit d'une généralisation anticipée, pensée pour un usage futur jamais concrétisé.

Dans les deux cas, cela mérite une relecture, car ce type d'abstraction inutile complique la compréhension sans apporter de bénéfice immédiat.

6.4.2 Paramètres de méthodes inutilisés

Red5 / io/.../main/java/org/red5/io/mp4/impl/MP4Reader.java

Remove this unused method parameter "scale". Why is this an issue? 2 years ago L584
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Remove this unused method parameter "scale". Why is this an issue? 2 years ago L683
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/.../main/java/org/red5/io/object/Serializer.java

Remove these unused method parameters. Why is this an issue? 2 years ago L74
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/src/main/java/org/red5/io/obu/OBUParser.java

Remove this unused method parameter "seq". Why is this an issue? 2 years ago L917
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Remove this unused method parameter "seq". Why is this an issue? 2 years ago L1802
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Remove this unused method parameter "seq". Why is this an issue? 2 years ago L1874
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/.../main/java/org/red5/io/utils/TlsUtils.java

Remove this unused method parameter "i". Why is this an issue? last year L141
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / server/.../java/org/red5/server/net/rtmp/InboundHandshake.java

Remove these unused method parameters. Why is this an issue? 4 years ago L425
Code Smell Major Open Not assigned 5min effort Comment cert, unused

8 of 8 shown

Ici c'est un peu plus particulier : il s'agit de méthodes qui sont appelées mais qui n'utilisent pas tous leurs paramètres.
L'analyse est plus complexe. On privilégie l'hypothèse selon laquelle le développeur ne s'en est pas rendu compte.

Cependant, cette interprétation doit être nuancée, car ce phénomène apparaît plusieurs fois à des endroits différents du projet, parfois avec le même paramètre. Il est donc possible qu'il existe une raison liée à la conception du système.

Pour l'instant, cette question reste ouverte et sera analysée plus en détail lors de la deuxième phase du projet.

7. Synthèse critique

L'étude de Red5-Server met en évidence un projet **solide, ancien et techniquement ambitieux**, dont la structure générale reste sérieuse malgré l'accumulation naturelle de dette technique.

Points forts

- projet toujours maintenu ;
- architecture globalement modulaire ;
- documentation présente et exploitable ;
- noms métier souvent cohérents ;
- usage structuré des pull requests ;
- forte spécialisation technique cohérente avec le domaine du streaming.

Points de vigilance

- documentation dispersée entre plusieurs fichiers ;
- couverture de test peu lisible et outillage à fiabiliser ;
- présence de classes très volumineuses ;
- complexité cyclomatique élevée dans certaines zones ;
- problèmes localisés de nommage, de shadowing et de paramètres inutilisés ;
- nombreux nombres magiques à expliciter davantage.